KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERNING

Course Name: Microprocessor and Microcontroller Laboratory

Course Code: CSE-2104

Project: VoteBox - IoT Digital Voting Machine

Dibbo Adhikary Roll: 2307080	Shrayashee Saha 2307079

Instructor: Md. Sakhawat Hossain Assistant Professor Department of CSE, KUET	Instructor: Md Tajmilur Rahman Lecturer Department of CSE, KUET

Table of Contents

1. Introduction

Elections form the foundation of democratic governance. Traditionally, elections have relied on paper ballots or manual counting systems, which are prone to human error, voter fraud, and delays in result declaration. As technology advances, the need for secure, efficient, and transparent voting systems becomes increasingly important — especially in institutional, organizational, and small-scale democratic processes such as school elections, club voting, and community polling.

This project presents an **IoT-Based Digital Voting Machine** that combines embedded hardware (Arduino microcontroller) with a modern browser-based real-time dashboard built using React.js. The system enables voters to cast votes using physical hardware — push buttons for voter identification — while simultaneously transmitting live vote data to a connected computer via serial communication. A browser dashboard called the **Election Command Center** visualizes results in real time using dynamic bar charts, candidate cards, and a live activity log.

The system handles multiple edge cases such as duplicate voting, invalid voter IDs, ties at first and second place, and closed election states — making it robust for real-world use. The combination of physical hardware security (voter ID switches) and software-level validation ensures integrity at both layers.

This project bridges the domains of **embedded systems**, **IoT communication**, and **modern web development**, demonstrating how low-cost microcontrollers like the Arduino can interface seamlessly with cutting-edge browser APIs such as the Web Serial API.

2. Features

Hardware Features (Arduino Side)

- **4-candidate voting** via dedicated push buttons (R1–R4)
- **Voter ID authentication** using a 4-bit DIP switch (supports up to 9 unique voters, IDs 1–9)
- **Duplicate vote prevention** — each voter ID can only vote once per session
- **LCD display (16×2 I2C)** for real-time feedback to the voter
- **Buzzer feedback system** with distinct tones:
    - Short double-beep ⟶ error (invalid input)
    - Long single-beep ⟶ warning (already voted, ID=0)
    - Short high-pitched beep ⟶ success (vote accepted)
- **VOTE button** to cast the selected vote
- **FINISH button** to close the election and display results
- **RESET button** to clear all votes and restart the session

- **Debounced button reading** to prevent false triggers
- **Serial data transmission** in structured format for dashboard parsing

Software / Dashboard Features (React Side)

- **Real-time vote visualization** via animated bar charts (Recharts)
- **Live candidate cards** showing vote count, percentage, and leading status
- **Winner/Tie detection** with animated result banners
- **Runner-up display** with tie-runner-up handling
- **Activity log** showing timestamped events
- **Demo/Simulation mode** for testing without hardware
- **Web Serial API integration** — direct browser-to-Arduino USB connection
- **Responsive dark-themed UI** with sci-fi aesthetic (Oxanium + Share Tech Mono fonts)
- **Scanline overlay and glow animations** for visual polish
- **No-backend architecture** — runs entirely in the browse

3. Hardware Required

Component	Quantity	Purpose
Arduino Uno (or Nano/Mega)	1	Main microcontroller
16×2 LCD with I2C Module (0x27)	1	Voter feedback display
Push Buttons	7	VOTE, FINISH, RESET + 4 candidate selectors
Push buttons	4	Voter ID input
Passive Buzzer	1	Audio feedback
10kΩ Pull-down Resistors	7	For button inputs
Breadboard	1–2	Prototyping
Jumper Wires (M-M, M-F)	~30	Connections
USB Type-B Cable	1	Arduino to PC (Serial + Power)
Computer with Chrome/Edge Browser	1	Running the React dashboard

4. Pin Assignment

Arduino Pin	Connected To
D2	ID Switch Bit 0 (LSB)
D3	ID Switch Bit 1
D4	ID Switch Bit 2
D5	ID Switch Bit 3 (MSB)
D6	Candidate R1 Button
D7	Candidate R2 Button
D8	Candidate R3 Button
D9	Candidate R4 Button
D10	VOTE Button
D11	FINISH Button
D12	RESET Button
D13	Buzzer
A4 (SDA)	LCD I2C SDA
A5 (SCL)	LCD I2C SCL
5V / GND	Power Rails

5. Arduino Code Logic

castVote()

This is the core voting function. It reads the voter ID, reads which candidate button is pressed, validates both, and either accepts or rejects the vote. It calls broadcastData("Voting") after every outcome (success or failure) so the dashboard stays synchronized.

showResults()

Called when FINISH is pressed. Implements a full result announcement sequence on the LCD, handling all edge cases: no votes, tie for 1st, single winner with no runner-up, runner-up tie, and single runner-up. Results are also sent via serial with `STATUS:Closed`.

doReset()

Resets all state: clears `voted[]` array, zeroes `votes[]`, sets `finished = false`, displays "Ready" on LCD, and broadcasts `STATUS:Ready` followed by `STATUS:Voting`.

waitOrReset(duration)

A blocking wait function that still monitors the RESET button during delay periods. If RESET is pressed within the wait window, it immediately calls `doReset()` and returns `true`, allowing callers to abort their current flow.

readID() and sampleID()

Implements a **majority-vote debounce**: reads the 4 ID pins three times with 3ms delays and returns the value that appears at least twice. This prevents noise on the pins from causing a wrong ID to be read.

readRep()

Scans all 4 candidate pins. Returns the index (0–3) only if **exactly one** is HIGH. If zero or multiple are HIGH, returns -1 (invalid).

Buzzer Patterns

- errorBeep() — two 150ms beeps at 800Hz (invalid input)
- warningBeep() — one 600ms beep at 500Hz (duplicate vote, ID=0)
- successBeep() — one 100ms beep at 1200Hz (vote accepted)

6. React Dashboard — Frontend Section

The browser-based Election Command Center is built as a single React component (`App.jsx`) with no backend dependency. It communicates directly with the Arduino via the **Web Serial API**, a modern browser capability that allows JavaScript to read from USB serial ports.

Key React Sections

Serial Communication (connectSerial())

- Requests a serial port from the browser using navigator.serial.requestPort()
- Opens at 9600 baud to match Arduino
- Uses TextDecoderStream to convert byte stream to text
- Reads lines, passes each to parseSerial(), and updates state

parseSerial(raw)

Parses the DATA|...|STATUS:... format:

- Splits by | to get vote segment and status segment
- Splits vote segment by , then by : to build a vote map
- Returns { votes: [r1, r2, r3, r4], status } or null if malformed

deriveResult(votes)

Pure function that computes election outcome from vote array:

- Returns { type: "no_votes" } if total is 0
- Returns { type: "tie_first", winners, maxVotes } if multiple candidates share the max
- Returns { type: "winner", winners, maxVotes, runnerUpType, runnerUps, secondVotes } otherwise
- runnerUpType is "none", "single", or "tie_runnerup"

ResultBanner Component

Conditionally renders one of three banner types when election is closed:

1. **No votes** — red alert banner
2. **Tie** — orange pulsing banner with trophy icons
3. **Winner** — gold glowing banner with full tally table and runner-up strip

Demo Mode

When toggled, a setInterval fires every 3 seconds, randomly incrementing one candidate's vote count and logging the event. This allows full UI testing without hardware.

Visual Design

- Dark space-themed palette (#060a12 background)
- CSS animations: cardIn, rowIn, blink, leadshine, tiepulse, scanpulse
- Candidate cards glow gold when leading, orange when tied
- Recharts BarChart with custom Cell coloring and tooltip
- Sticky header with connection status indicator

7. Circuit / Wiring Diagram

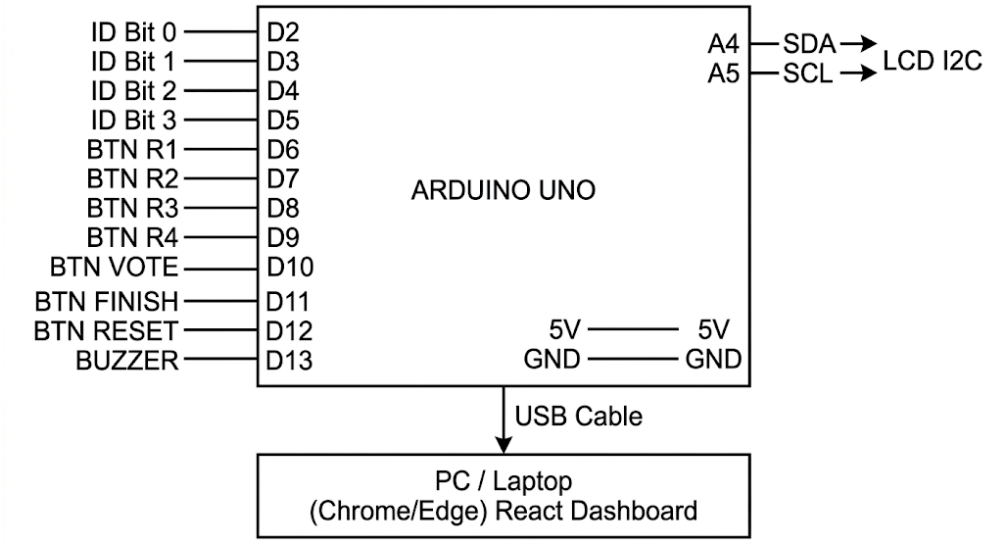

Fig:Pin diagram

Tinkercad Implementation

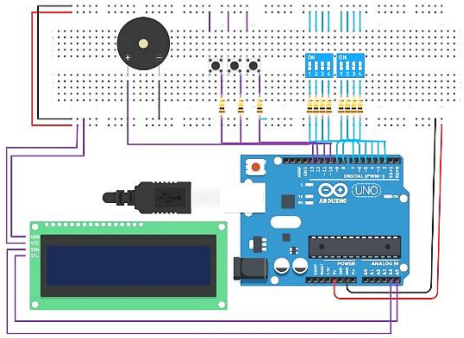

Fig: Voting machine with Arduino

8. System Flowchart

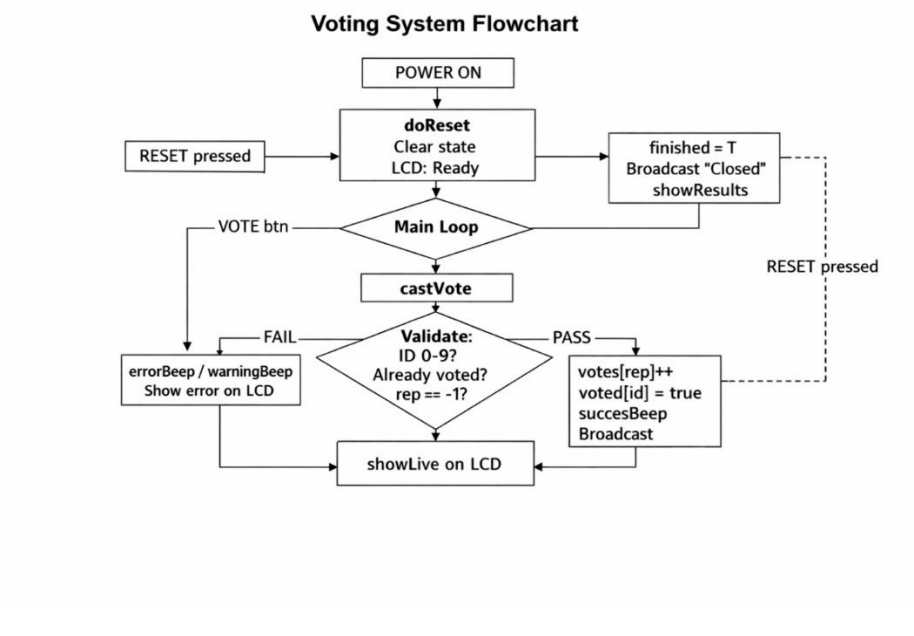

Fig: Flow chart for Voting machine

9. Limitations

1. **Maximum 9 voters** — The 4-bit DIP switch supports IDs 1–9 only. Scaling to more voters requires a different ID input method (e.g., RFID, keypad).
2. **No physical security** — Buttons are exposed; a voter could physically press a candidate button without proper oversight. A ballot-box style enclosure is needed for real-world use.
3. **Single-session memory** — All votes are stored in Arduino RAM. A power cut before FINISH is pressed loses all data. No EEPROM or SD card persistence is implemented.
4. **Chrome/Edge only** — The Web Serial API is not supported by Firefox, Safari, or most mobile browsers, limiting the audience for the dashboard.
5. **No encryption** — Serial data is transmitted as plain text. A man-in-the-middle could intercept or modify data if the USB connection is compromised.
6. **Fixed 4 candidates** — The system is hardcoded for exactly 4 representatives. Adding or removing candidates requires code changes.
7. **Single Arduino connection** — The dashboard can connect to only one Arduino at a time; multi-station voting is not supported.
8. **No voter authentication beyond ID** — Anyone who knows or guesses an unused voter ID could impersonate that voter. There is no biometric or password verification.
9. **LCD is local only** — The LCD display is only visible to the person physically at the device; remote voters are not supported.
10. **Requires installation knowledge** — Setting up the circuit and uploading Arduino code requires technical skills not available to all users.

10. Social Impact

Promoting Democratic Participation

By making voting systems more accessible and transparent, projects like this encourage participation in small democratic processes — club elections, classroom votes, community decisions — where traditional ballot systems are often skipped due to complexity.

Educational Value

This project serves as a hands-on learning platform for students studying electronics, IoT, and web development. It bridges the gap between hardware programming and modern frontend development, teaching integrated system design.

Reducing Election Fraud at Small Scale

The duplicate-vote prevention mechanism and structured ID validation reduce the most common form of fraud in

informal elections: voting more than once. Real-time transparency via the dashboard also makes manipulation visible immediately.

Cost-Effective Digital Governance
For schools, NGOs, and local organizations in developing regions, a $20 Arduino-based voting system with a free browser dashboard makes digital elections financially viable where commercial e-voting solutions are prohibitively expensive.

Transparency and Trust
Real-time result broadcasting to a public display builds trust in the process. Voters can watch the live vote count update as each vote is cast, removing the "black box" nature of many electronic voting systems.

Potential Concerns

- If deployed without proper physical security, the system could be manipulated at the hardware level.
- Over-reliance on a single USB-connected computer creates a single point of failure.
- Not suitable for national or large-scale elections without significant security enhancements.

11. Future Improvements

1. **RFID/NFC Voter Cards** — Replace DIP switches with RFID card readers for more voters and stronger identity verification.
2. **EEPROM / SD Card Storage** — Persist votes to non-volatile memory so data survives power loss. An SD card module would allow storing thousands of voter records.
3. **Wi-Fi Dashboard (ESP8266/ESP32)** — Replace the USB serial link with a Wi-Fi module, allowing the dashboard to run on any device on the local network — no cable needed.
4. **Multi-Station Support** — Allow multiple Arduino voting booths to connect to a central server, aggregating results from all stations in real time.
5. **Encrypted Communication** — Implement a simple HMAC or AES encryption layer over the serial protocol to prevent data tampering.
6. **Voter Receipt System** — Print or display a unique encrypted receipt for each vote cast, enabling post-election audit without revealing who voted for whom.
7. **Biometric Authentication** — Integrate a fingerprint sensor module for voter authentication, eliminating the need for external ID cards entirely.
8. **Mobile-Responsive Dashboard** — Adapt the React dashboard for mobile browsers using a WebSocket connection (via an ESP32 server), enabling live monitoring on smartphones.
9. **Multi-Language LCD Support** — Add support for regional language characters on the LCD display to improve accessibility.

10. **Automatic Timer** — Add a countdown timer that automatically closes the election after a set duration, removing the need for a FINISH button press.
11. **Admin Authentication** — Require a PIN code or admin RFID card to press FINISH or RESET, preventing unauthorized election termination.
12. **Database Integration** — Store results in a cloud database (Firebase, Supabase) for long-term record keeping and historical comparison.

12.COMPARISON WITH EXISTING PROJECTS

The biometric-based Electronic Voting Machine proposed by **V. Kiruthika Priya along with V. Vimaladevi, B. Pandimeenal, and T. Dhivya was published in the International Conference on Trends in Electronics and Informatics (ICEI 2017) in the year 2017.** Their project mainly focused on improving voting security using a fingerprint sensor connected with Arduino UNO. The system authenticates voters through biometric verification before allowing vote casting and displays results on an LCD screen. Although the project improves voter authentication and reduces manual fraud, the system remains mostly hardware-oriented with limited visualization, no real-time monitoring dashboard, no live analytics, and no IoT-based election supervision features.

Similarly, the Arduino UNO based Electronic Voting Machine developed by **Debashish Dash, Dileesha A, Jenifer Shanmugasundaram, Sangeetha M, and Sharon Jessika S was published as a ResearchGate project in recent years**. Their work mainly focused on replacing traditional ballot voting using Arduino UNO, push buttons, LEDs, and LCD display modules. The system performs digital vote counting and displays results electronically, but its implementation is comparatively simple and limited to embedded hardware operation. The project lacks advanced functionalities such as real-time IoT communication, intelligent edge-case handling, live graphical monitoring, serial communication dashboards, automated election logs, and dynamic visualization of election statistics.

In contrast, the proposed IoT Digital Voting System combines embedded hardware with modern software technologies to create a far more advanced election management platform. Unlike the earlier systems, the proposed project integrates Arduino hardware with a React-based Election Command Center using Web Serial API communication for real-time vote synchronization and monitoring. The system provides live graphical vote distribution, animated dashboards, automated activity logs, winner and runner-up analysis, tie-condition detection, and intelligent handling of edge cases such as duplicate voting, invalid voter IDs, multiple candidate selection, and voting after election closure. Therefore, compared to both previously published systems, the proposed project delivers higher transparency, better scalability, stronger fault handling, improved visualization, and a more professional IoT-enabled smart voting environment suitable for modern digital election systems.

Also Two Arduino-based Electronic Voting Machine (EVM) implementations are commonly found in online tutorials such as https://www.youtube.com/watch?v=BkKHqqIz9Bo titled “Electronic Voting Machine using Arduino” and https://www.youtube.com/watch?v=a7SsWAD85Lo titled `”Smart Electronic voting maching using Arduino”Both systems mainly demonstrate basic vote casting using push buttons, Arduino UNO, and a 16×2 LCD, focusing on simple counting

mechanisms without strong authentication or advanced control logic. In contrast, the proposed system in this report uses a structured polling workflow with a master control switch, real-time vote monitoring, and systematic result display, making it more organized and closer to an institutional voting model. Unlike the YouTube implementations that are primarily educational prototypes, this design emphasizes controlled operation, improved reliability, and better system-level management of the voting process.

13. Conclusion

This project successfully demonstrates the design and implementation of a complete IoT-based digital voting system that integrates embedded hardware, serial communication, and a modern real-time web dashboard. The Arduino-based hardware handles voter authentication, vote recording, duplicate prevention, and result computation, while the React.js dashboard provides a professional, visually rich interface for monitoring the election in real time.

The system rigorously handles ten distinct edge cases — including zero-vote elections, first-place ties, runner-up ties, invalid voter IDs, duplicate voting attempts, and post-closure vote attempts — making it far more robust than a simple prototype. The dual-layer validation (hardware debouncing + software logic) and structured serial protocol ensure reliable data transfer between the embedded and web layers.

From a technical standpoint, the project showcases the practical integration of embedded C++ programming, the Arduino ecosystem, the modern Web Serial API, and React's state-driven UI paradigm. From a social standpoint, it offers a low-cost, transparent, and accessible voting solution suitable for educational institutions, community organizations, and civic technology initiatives.

While current limitations prevent its use in large-scale or high-security elections, the architecture is extensible. With additions such as RFID authentication, Wi-Fi connectivity, and encrypted communication, the system could evolve into a genuinely viable small-scale e-voting platform. As digital civic infrastructure becomes increasingly important worldwide, projects like this play a meaningful role in demonstrating what accessible, open, and transparent democratic technology can look like.

14. References

1. Arduino. (2024). *Arduino Uno Rev3 Documentation*. Arduino Official Website. https://docs.arduino.cc/hardware/uno-rev3
2. Arduino. (2024). *LiquidCrystal I2C Library Reference*. Arduino Reference. https://www.arduino.cc/reference/en/libraries/liquidcrystal-i2c/
3. Arduino. (2024). *tone() Function Reference*. Arduino Reference. https://www.arduino.cc/reference/en/language/functions/advanced-io/tone/